

X. La cascade et l'héritage — Comprendre la priorité des styles

LA CASCADE EN UNE PHRASE

Quand deux règles CSS sont en conflit sur un même élément, la **CASCADE** détermine laquelle gagne grâce à 3 critères : l'importance, la spécificité, et l'ordre.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Les 3 critères de la cascade**
 - Exemple pas à pas
- **B. La spécificité**
 - 1. Les 4 niveaux de spécificité
 - 2. Comment calculer
 - 3. Exemples de calcul
 - 4. Éviter les IDs en sélecteur
- **C. L'héritage**
 - 1. Le principe
 - 2. Propriétés héritées vs non héritées
 - 3. Pourquoi background-color ne s'hérite pas ?
- **D. Les mots-clés inherit, initial, unset, revert**
 - 1. inherit
 - 2. initial
 - 3. unset
 - 4. revert
- **E. !important : l'arme nucléaire**
 - 1. Ce que ça fait
 - 2. Pourquoi l'éviter
 - 3. Les (rares) cas légitimes
 - 4. L'ordre exact de la cascade complète
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Expliquer** les 3 critères de la cascade (importance, spécificité, ordre).
- **Calculer** la spécificité d'un sélecteur sur l'échelle 1 / 10 / 100 / 1000.
- **Identifier** les propriétés qui **s'héritent** (texte, couleur) vs celles qui ne s'héritent pas (box-model).
- **Utiliser** `inherit`, `initial`, `unset`, `revert` à bon escient.
- **Diagnostiquer** un conflit CSS dans les DevTools.

2. En pratique — situation professionnelle

Vous écrivez `.btn { color: white; }` mais votre bouton reste noir. Après 20 minutes de recherche, vous découvrez qu'une règle `#app div button { color: black; }` écrase la vôtre. Sans maîtrise de la cascade, vous ajoutez un `!important` pour *forcer* — mauvaise idée. Cette leçon vous apprend à diagnostiquer et résoudre ces conflits **proprement**.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Diagnostiquer un style qui ne s'applique pas (DevTools → barré)	Empiler <code>!important</code> pour gagner contre la cascade
<code>inherit</code> sur les liens pour suivre la couleur du parent	Sélectionner par <code>#id</code> pour styliser (spécificité 100)
Classes BEM peu spécifiques pour rester surchargeable	Surcharger Bootstrap avec <code>!important</code> sans réfléchir

A. Les 3 critères de la cascade

Quand plusieurs règles tentent d'appliquer des styles différents au même élément, le navigateur décide **laquelle gagne** en appliquant 3 critères **dans cet ordre** :

1. **Importance** — **!important** > normal.
2. **Spécificité** — un sélecteur plus précis gagne sur un sélecteur plus vague.
3. **Ordre d'apparition** — si tout est égal, la **dernière règle déclarée** gagne.

Exemple pas à pas

```
/* Règle 1 */
p {
  color: blue;
}

/* Règle 2 — plus spécifique */
.intro {
  color: green;
}

/* Règle 3 — même spécificité que la 2, mais déclarée après */
.intro {
  color: red;
}
```

```
<p class="intro">Bonjour</p>
```

Analyse :

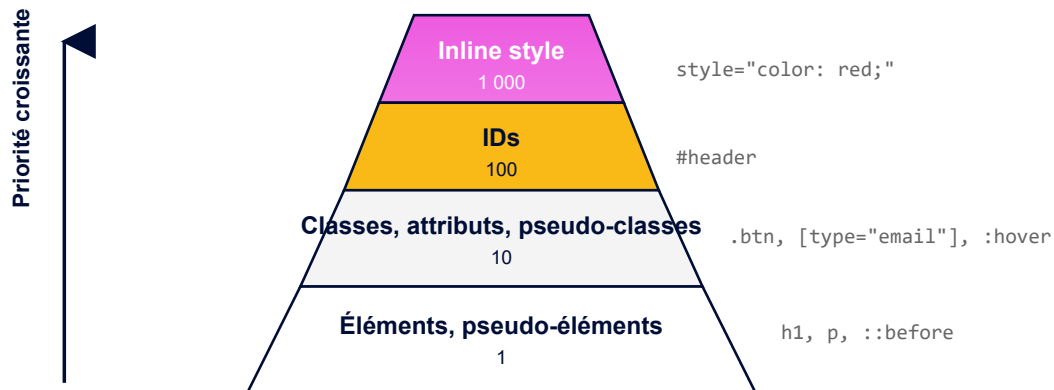
- Règle 1 : spécificité 1 (type `p`). Perd face à la 2 (spécificité 10).
- Règle 2 vs 3 : même spécificité (10). Ordre d'apparition tranche → **règle 3 gagne**.
- **Résultat** : texte **rouge**.

◆ Diagnostiquer dans les DevTools

Inspecter un élément → panneau de droite *Styles*. Chrome affiche les règles dans l'ordre de priorité, avec les déclarations écrasées **barrées**. C'est le moyen le plus rapide de comprendre pourquoi un style ne s'applique pas.

B. La spécificité

1. Les 4 niveaux de spécificité



!important
écrase tout — à éviter

2. Comment calculer

Lire le sélecteur et compter :

- combien d'**IDs** (`#xxx`) → centaines ;
- combien de **classes** (`.xxx`), **attributs** (`[xxx]`), **pseudo-classes** (`:xxx`) → dizaines ;
- combien de **types** (`div` , `p`) et **pseudo-éléments** (`::before`) → unités.

3. Exemples de calcul

Sélecteur	Calcul	Score
<code>p</code>	1 type	0,0,1
<code>.btn</code>	1 classe	0,1,0
<code>ul li.active</code>	2 types + 1 classe	0,1,2
<code>#header</code>	1 ID	1,0,0
<code>#header .menu a:hover</code>	1 ID + 2 classes/pseudo + 1 type	1,2,1
<code>style="..."</code> (inline)	n/a, supérieur à tout	~1,0,0,0

⚠ La spécificité n'est pas une addition décimale

Un sélecteur de spécificité `0,1,0` (une classe) **perd toujours** face à `1,0,0` (un ID), **peu importe** la quantité de classes. Même 11 classes (`0,11,0`) perdent face à un ID (`1,0,0`). Le dièse `#` est vraiment puissant.

4. Éviter les IDs en sélecteur

Parce qu'un ID a une spécificité très élevée, il est **difficile d'écraser** son style sans recourir à **!important**. Règle du pouce :

- **Classes pour les styles** ✓
- **IDs pour JavaScript / ancres / labels** ✓
- **IDs pour styliser** ✗

Si vous avez besoin d'un ID (unicité sémantique), utilisez-le mais ciblez-le par une classe équivalente en CSS :

```
<header id="main-header" class="site-header">
```

```
.site-header { background: navy; } /* spécificité 10, faible, facile à surcharger */
```

C. L'héritage

1. Le principe

Certaines propriétés CSS sont **transmises automatiquement** des parents aux enfants. D'autres non.

```
body {
  color: #333;
  font-family: Arial;
}
```

→ Tous les descendants de `<body>` (`<p>`, `<div>`, `<span>`, `<a>`, ...) héritent de **color** et **font-family** sans qu'on ait à le spécifier.

2. Propriétés héritées vs non héritées

Héritées (texte, typo)	Non héritées (layout, box)
color	margin , padding , border
font-family , font-size , font-weight , line-height	width , height
text-align , text-indent	background (exception notable)
visibility	display
cursor	position , top , left
list-style	box-shadow

Règle mnémotechnique : **tout ce qui concerne le texte s'hérite**, tout ce qui concerne la **boîte** et la **position** ne s'hérite pas.

3. Pourquoi `background-color` ne s'hérite pas ?

Parce qu'on veut pouvoir donner une couleur de fond à un conteneur parent **sans** que ses enfants aient automatiquement la même couleur (leur texte aurait des fonds redondants). Par défaut, `background-color` est **transparent** pour les enfants — ils **laissent voir** le fond du parent, ce qui donne l'illusion d'héritage mais n'en est pas techniquement.

D. Les mots-clés `inherit`, `initial`, `unset`, `revert`

Ces 4 mots-clés marchent sur **n'importe quelle propriété CSS**.

1. `inherit`

Force l'héritage de la valeur du parent, même pour une propriété non héritée par défaut.

```
.lien {  
  color: inherit;      /* prend la couleur du parent (utile dans une nav) */  
}
```

2. `initial`

Rétablit la **valeur initiale** de la propriété (celle de la spec CSS, pas celle du navigateur).

```
.reset {  
  color: initial;      /* = black */  
  font-size: initial;  /* = medium (généralement 16px) */  
}
```

3. `unset`

Combine les deux :

- **Pour une propriété héritée** : se comporte comme `inherit`.
- **Pour une propriété non héritée** : se comporte comme `initial`.

4. `revert`

Rétablit la valeur que le **navigateur** applique par défaut (user-agent stylesheet).

```
a {  
  text-decoration: revert; /* rétablit le soulignement natif des liens */  
}
```


Quand les utiliser ?

- `inherit` : forcer une propriété non héritée (ex: `color: inherit` sur les liens pour qu'ils suivent la couleur du texte parent).
- `initial` / `unset` / `revert` : resets locaux dans des composants isolés (ex: reset de bouton pour l'écrire from-scratch).

E. **!important** : l'arme nucléaire

1. Ce que ça fait

Ajouté à une déclaration, **!important** lui donne la **priorité absolue** dans la cascade, indépendamment de la spécificité et de l'ordre.

```
.btn {  
  color: red !important;  
}  
  
#app div button {  
  color: blue;    /* perdu, malgré spécificité plus haute */  
}
```

2. Pourquoi l'éviter

!important **casse la cascade naturelle**, ce qui produit des effets en cascade (jeu de mots) :

- **Plus possible de surcharger** sans empiler des **!important** partout — guerre de l'escalade.
- **Code difficile à maintenir** — l'ordre de priorité devient imprévisible.
- **Symptomatique d'un problème de structure CSS** — si vous en avez besoin, c'est que vos sélecteurs sont trop spécifiques ailleurs.

3. Les (rares) cas légitimes

- **Overrides ciblés** d'un framework externe (Bootstrap, Tailwind) quand vous ne pouvez pas modifier sa source.
- **Styles utilitaires ultimes** : `.hidden { display: none !important; }` — garantir que la classe fait son boulot quoi qu'il arrive.
- **Styles d'impression** : `@media print { body { background: white !important; } }`.

4. L'ordre exact de la cascade complète

Pour être précis, voici l'ordre **complet** appliqué par le navigateur (du plus prioritaire au moins prioritaire) :

1. Styles **!important** de l'utilisateur final (user stylesheet)
2. Styles **!important** du site (auteur)
3. Styles normaux du site (auteur)
4. Styles normaux de l'utilisateur final
5. Styles **!important** du navigateur (user agent)
6. Styles normaux du navigateur

Dans 99 % des cas, vous travaillez uniquement dans les zones 2 et 3.

Conclusion

1. À retenir

- **Cascade = 3 critères** : importance > spécificité > ordre d'apparition.
- **Spécificité** : score à 3 niveaux (0,0,1 jusqu'à 1,0,0), les IDs dominent toujours les classes.
- **Héritage** : les propriétés de **texte** (color, font-*) s'héritent ; celles de **boîte** (margin, padding, width) ne s'héritent pas.
- Pour forcer : `inherit`, `initial`, `unset`, `revert` — utiles pour resets locaux.
- **!important** = **dernier recours**, pas solution par défaut. Si vous en abusez, revoyez vos sélecteurs.
- Les **DevTools** montrent la cascade en temps réel : toujours les consulter pour diagnostiquer.

2. Pour aller plus loin

- **MDN — Cascade et héritage**
- **MDN — Spécificité**
- **MDN — inherit / initial / unset / revert**
- **CSS Tricks — Specificity Calculator**